

Authors:

Marwan Al-Jaufy
Amr Alhamwi
Mustafa Syed
Farhan Panchbhaya

Contributors:

Marwan Al-Jaufy
Amr Alhamwi
Mustafa Syed
Farhan Panchbhaya

Last updated: 10/06/2026

Status: <Draft.>

Project Design Document

Purpose

We chose a modular monolithic architecture with a package by feature type of organization and a shared kernel. This architecture:

1. Works harmoniously with Vaadins server side user interface and WebSocket push.
2. It enables us to implement distributed systems pattern (Saga, Outbox, CQRS, Idempotency, Circuit Breaker) without accidental network complexity
3. It facilitates team parallelism through clear package boundaries whereby each team member owns a feature package whether it is User, Posts, Voting or Notifications.
4. It helps us minimize coordination overhead in that, Git merges replace API contract negotiations.
5. This architecture provides a clear extraction path to microservices if future scale demands it
6. It also follows industry best practices where the modular monoliths successfully support large teams

We did consider microservices as it offers independent service deployment which would be a major plus to our project. However, due to our team size, course timelines and Vaadin constraints, the modular monolithic architecture was seen to be the most appropriate choice.

Background Reading

Some of the external resources that we used to better understand the concepts that underline the chosen design are mentioned below:

Resource	Description	Relevance
"The Monolith Strikes Back: Why?" (ITDS,2025)	It provides an industry analysis documenting how Amazon, Shopify, and Segment are reversing course from microservices back to modular monoliths. It reports on the impact of Amazon Prime Video reducing infrastructure costs by over 90% after moving away from microservices	This document supports our architectural choice by providing real world evidence that modular monoliths are a mature pragmatic choice
"Beyond the Hype: When Microservices Beat (Modular) Monoliths in 2025" (University of Tampere 2025)	This is a systematic literature review (51 academic/industry sources, 2020-2025) proposing a decision framework for architecture selection based on team cognitive load, platform maturity, and workload characteristics.	This provides an academic validation that the team size and organizational capacity should guide architectural decisions and not trends. Our team size aligns with the recommendation for a modular monolith
"Modular monolith and 'package by component'" (Simon Brown 2016)	This is a seminal blog post (republished from "Coding the Architecture"). This critically compares package by layer, package by feature, ports & adapters. The author's proposed "package by component" approach. It discusses the limitations of layered architectures and the need for compiler enforced modular boundaries.	This was a foundational reading for the package by feature organization. Explains why horizontal layering alone leads to architectural erosion over time
"Server Push Configuration" (Vaadin Docs, v24)	This is the official Vaadin documentation for enabling @Push, push modes (automatic,manual), WebSocket transport configuration, and the access() method pattern for thread safe user interface updates.	This was a required reading for us when it comes to knowing about the implementation of features. It explains the broadcaster pattern for cross session updates central to our collaborative Reddit clone

Context

Our project involves building an advanced Reddit-like social platform that has real time posts, comments, voting, subreddits, and notifications. The end users should be able to:

1. Create accounts and join topic based communities (Subreddits)
2. Create text, link, or image posts within communities
3. Comment on posts in threaded discussions
4. Upvote or downvote posts and comments (affecting content visibility)
5. View feeds sorted by "hot", "new", "top", or "controversial"

6. Receive real time updates (live vote counts, instant comment notifications)

And as a project based in the Advanced Software Engineering course, the platform has to demonstrate:

1. Real time collaborative features (live vote counts, instant comments)
2. Distributed systems patterns (Saga, Outbox, Idempotency)
3. Team based development (multiple developers working in parallel)
4. Production ready deployment (Docker, monitoring, testing)

The chosen technology constraints are: Java, Spring - Framework, Web based delivery.

Another chosen framework is: Vaadin which will provide server side UI with WebSocket push.

Project Business Goals

Goal	Target Stakeholder	How the Platform Provides it
Enable community discussions	End User (readers, posters)	Users can create subreddits, post content and engage in threaded comment discussions
Surface quality content	End User (voters, readers)	Upvote/downvote system with karma scoring ranks popular content; feed sorting (hot, top, new) helps users discover relevant posts
Foster real time engagement	End Users (active participants)	Live vote counters and instant comment updates (Vaadin Push) create a responsive, social experience without page refreshes
Allow self moderation	Community moderators	Users with sufficient karma can moderate their subreddits (delete posts, remove comments, ban users)
Reduce information overload	End Users (readers)	Subreddit subscriptions allow users to curate their personal feed; search functionality finds relevant posts and comments
Support community growth	Platform owners (hypothetical)	Event driven architecture (Kafka,outbox) enables future features like recommendation engines, analytics and email digests without rewriting core logic
Maintain trust & safety	Platform owners, moderators, Users	Optimistic locking prevents vote manipulation; idempotent event handling ensures actions are not double counted; circuit breakers prevent cascading failures

Provide a reference for implementation	Course Instructor,	Clean modular monolith with package by feature demonstrates advanced patterns (Saga, CQRS, Outbox) in a realistic social platform
--	--------------------	---

How the Architecture enables the above business goals:

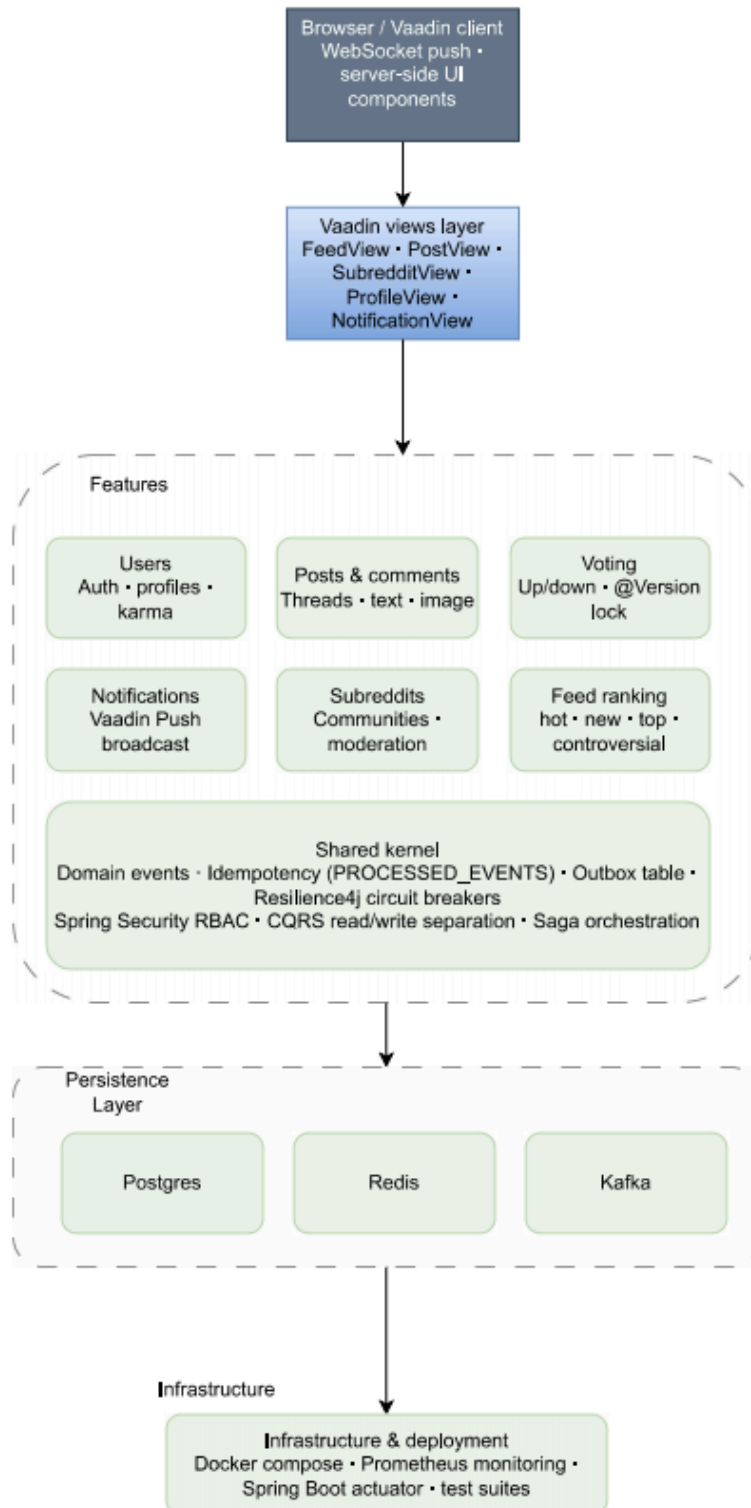
Business Goal	Architectural Feature that Enables it
Real time engagement	Vaadin Push (WebSocket) with broadcaster pattern
Quality content ranking	CQRS + Redis materialised views for hot feed calculation
Self moderation	Spring Security role based access
Reducing information overload	PostgreSQL full text search, subreddit subscription model
Community growth/future features	Transactional Outbox + Kafka for reliable event streaming
Trust & Safety	Optimistic locking (@Version), idempotent handlers (PROCESSED_EVENTS table), Resilience4j circuit breakers
Reference implementation	Package by feature organisation, clear architectural documentation

Prior work that led to this design:

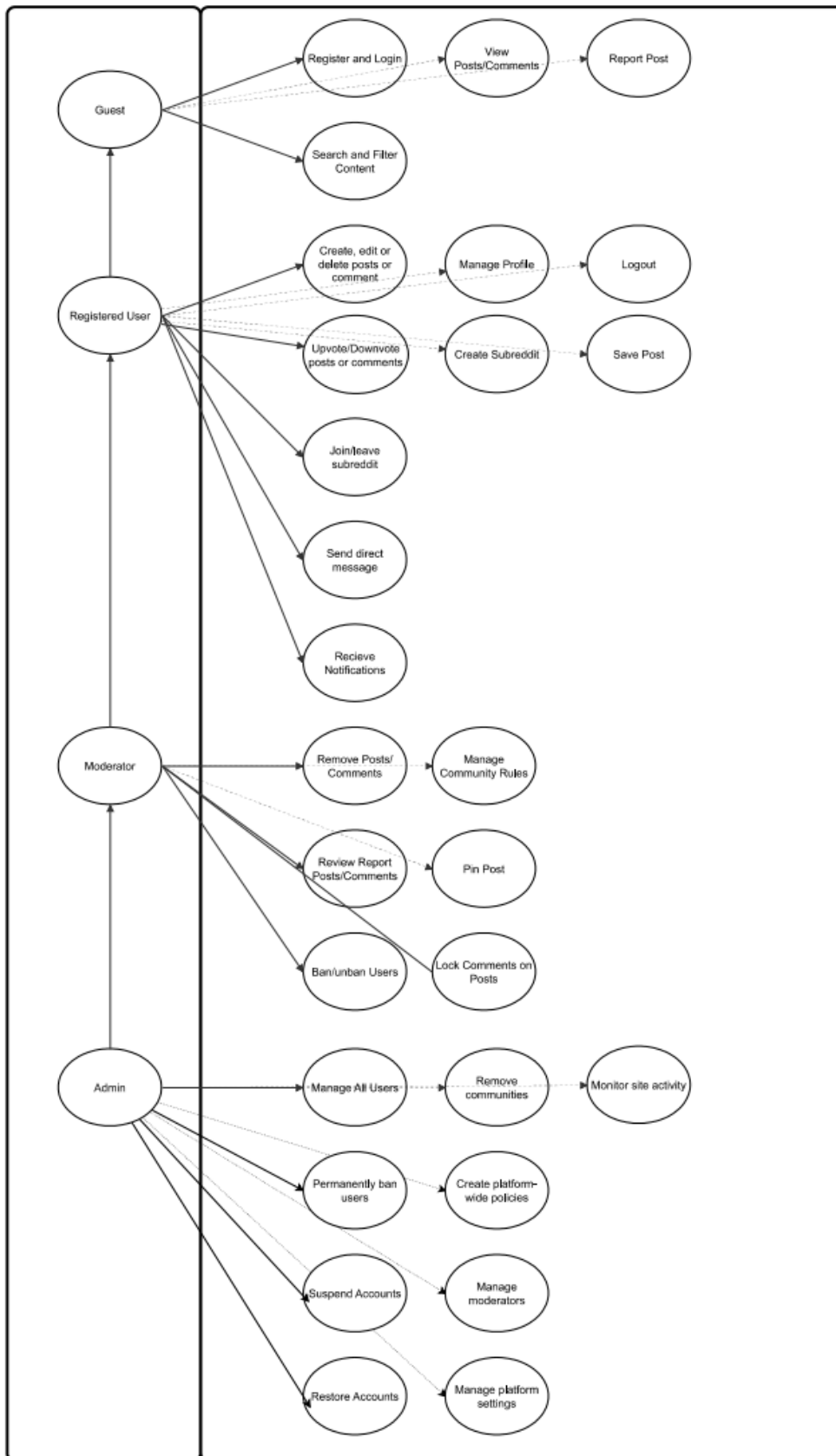
Prior Work	What we learned	Applied in our design
Reddit.com (production platform)	Real time voting and comments require live updates; feed ranking needs efficient caching	Vaadin Push + Redis materialised views
Hacker News (open source clone attempts)	Naive upvote/downvote without optimistic locking leads to vote manipulation	@Version on Vote entity prevents double counting
Spring Petclinic (monolithic reference)	Clean separation of concerns (controller -> service -> repository) is maintainable	Adopted layered architecture within each feature
Amazon Prime Video (microservices -> monolith)	Distribution adds accidental complexity; start simple and extract when needed	Modular monolith with extraction path to microservices

Detailed Design

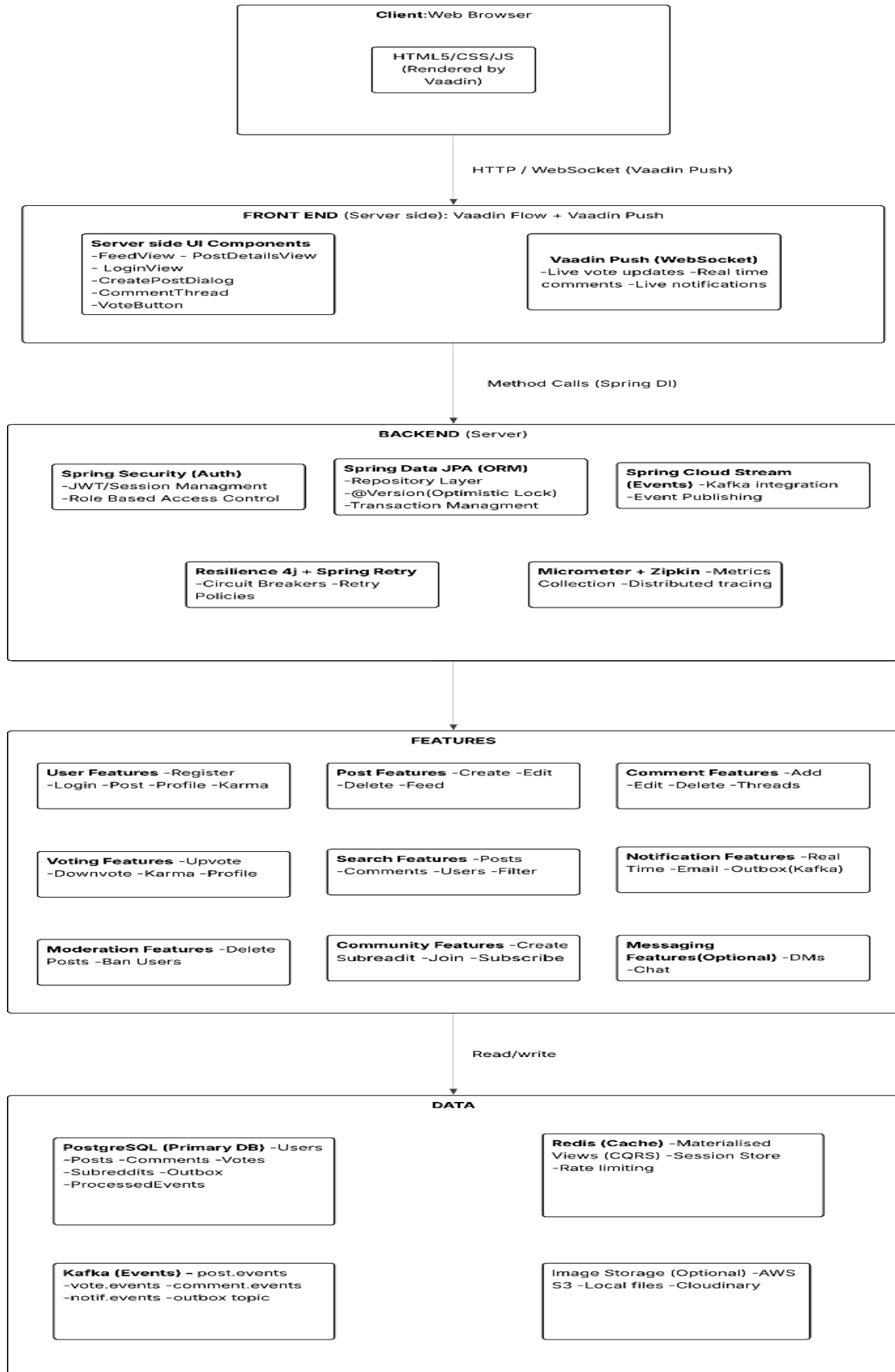
Overall System Architecture



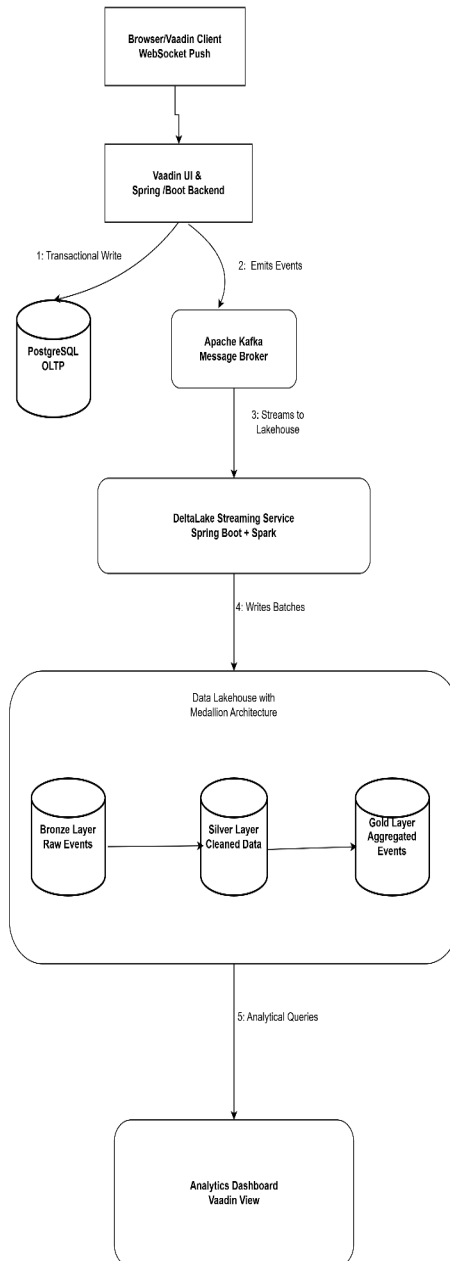
UML Use Case Diagram



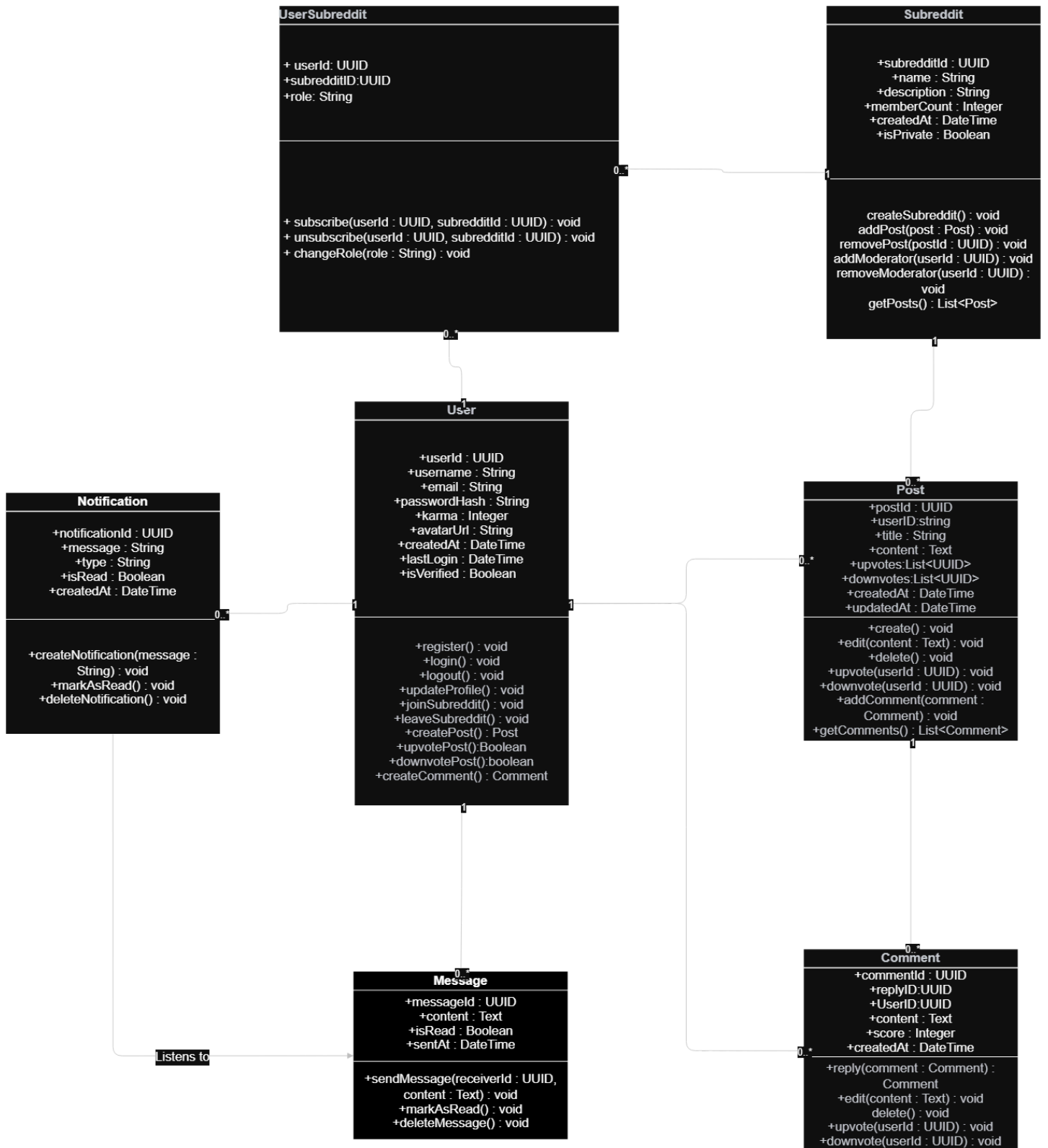
UML Component Diagram



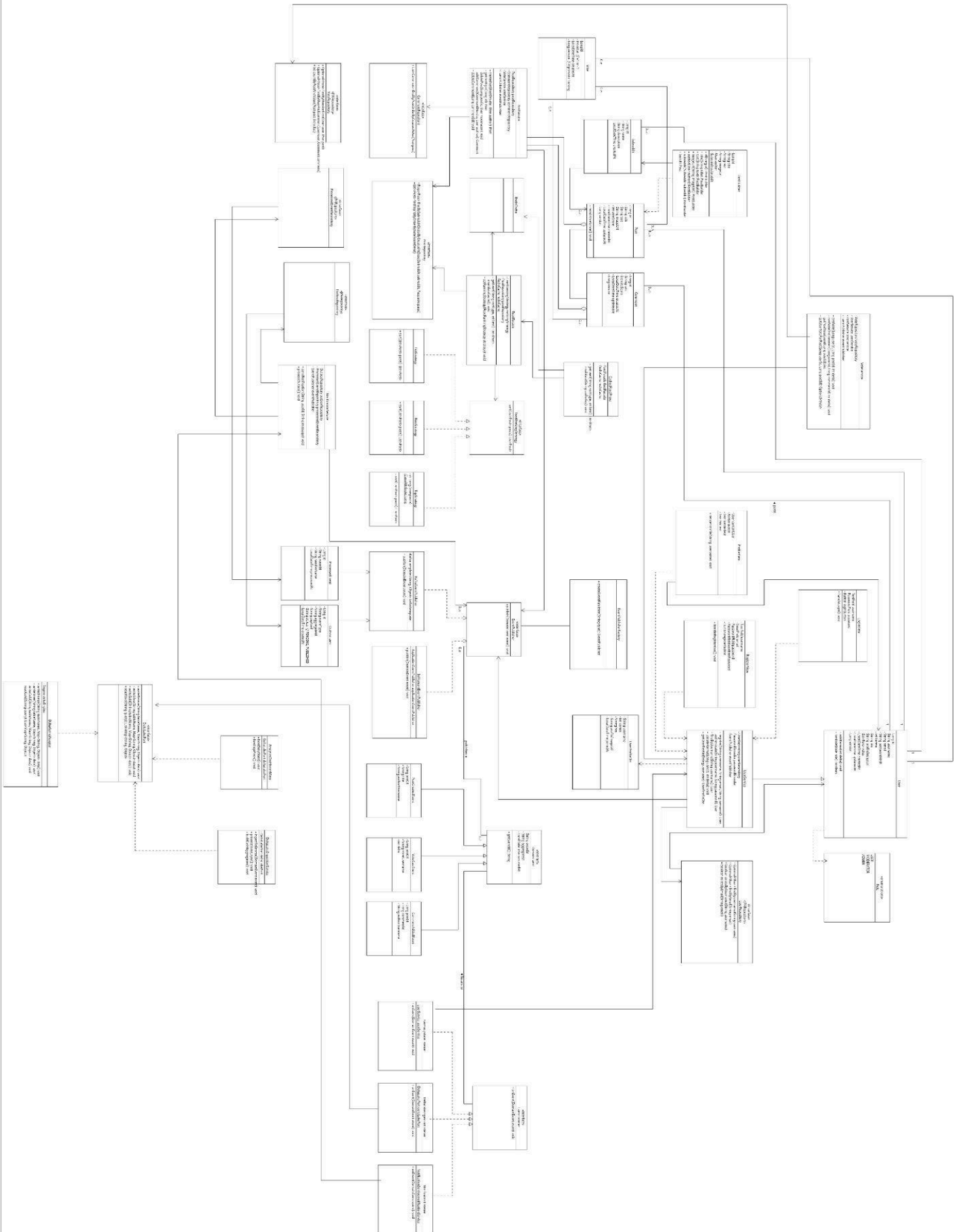
Data flow diagram



UML Class Diagram

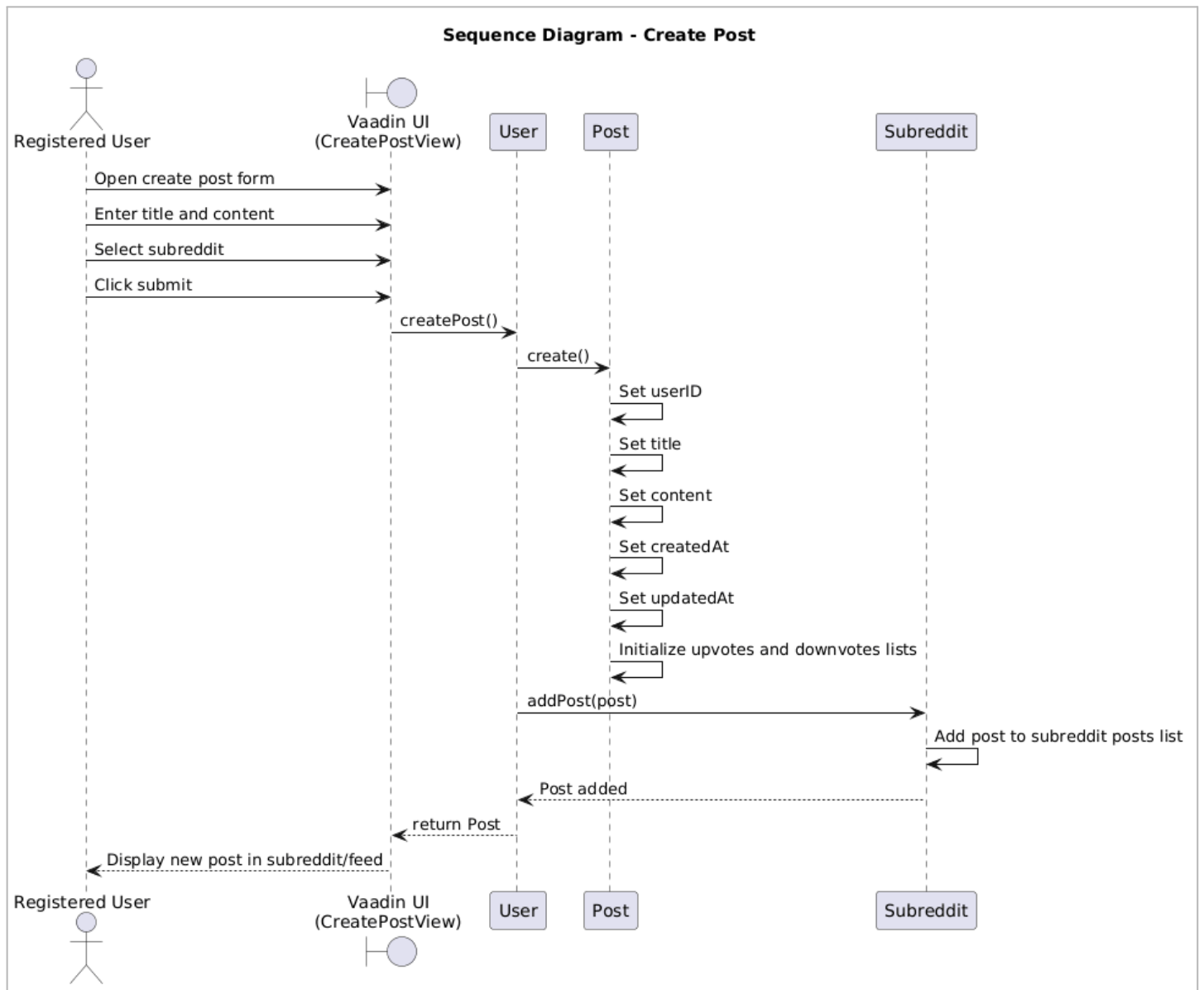


Updated UML Class Diagram (21-06-2026)

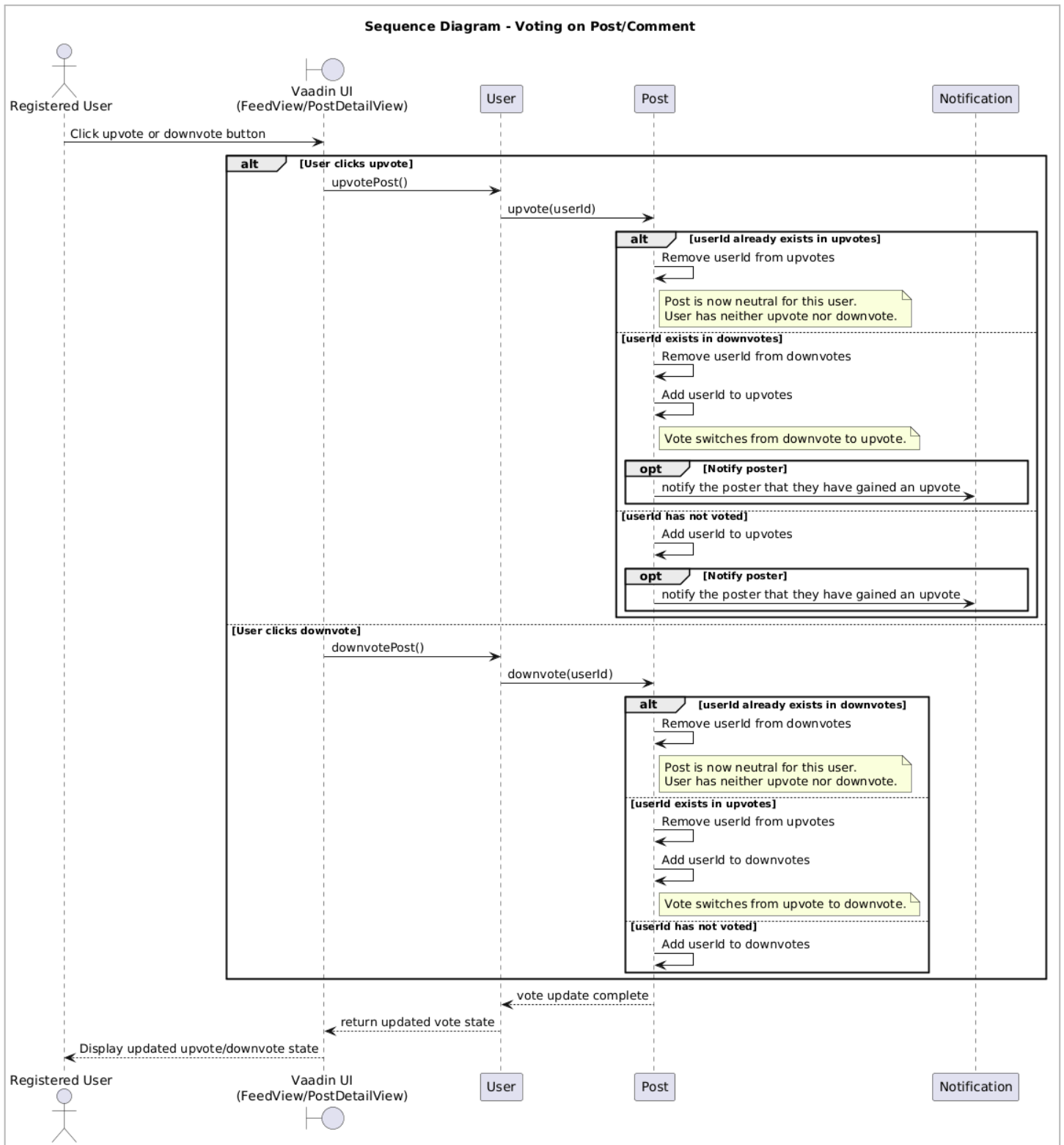


Sequence Diagrams

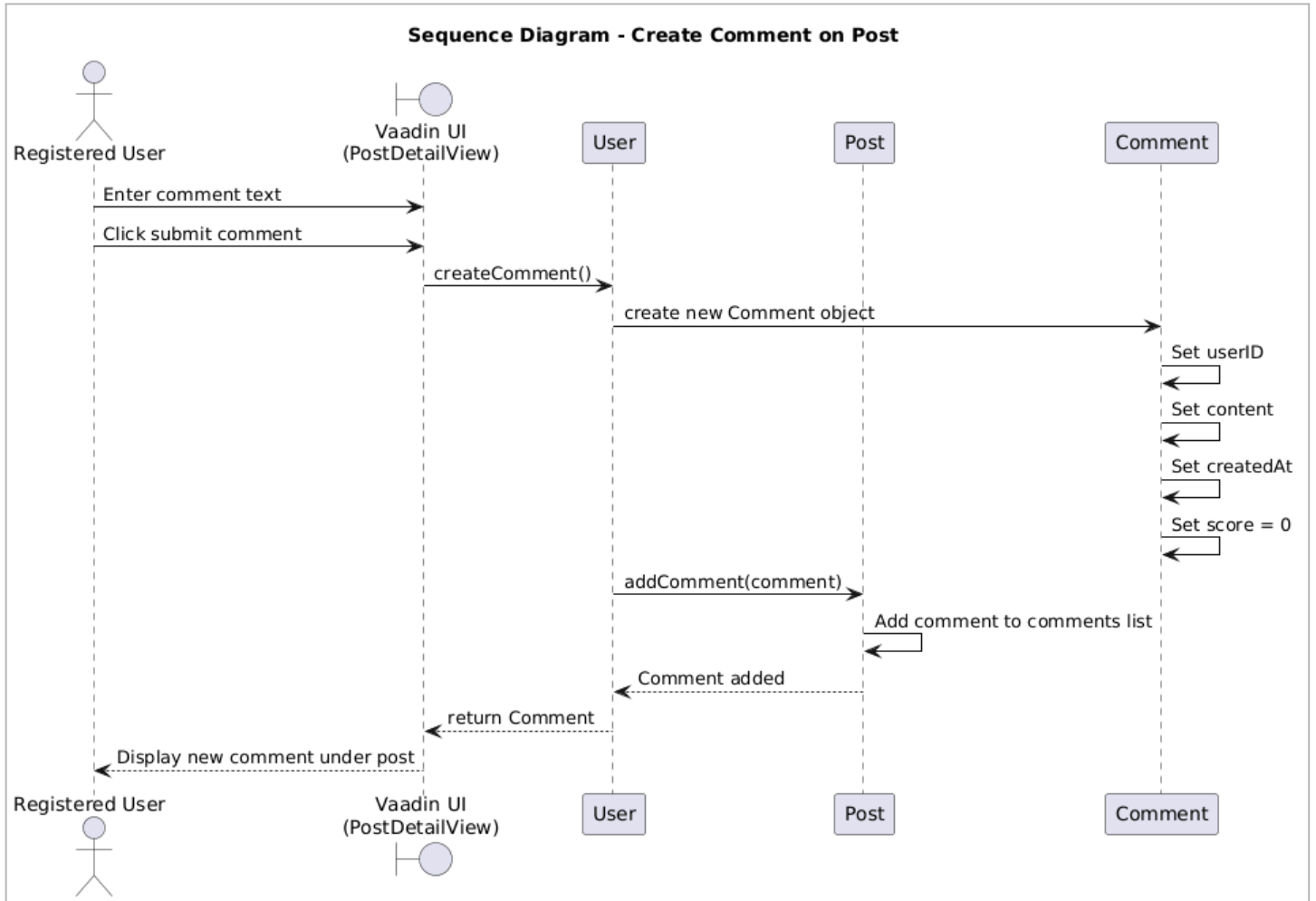
Creating a post



Voting sequence diagram

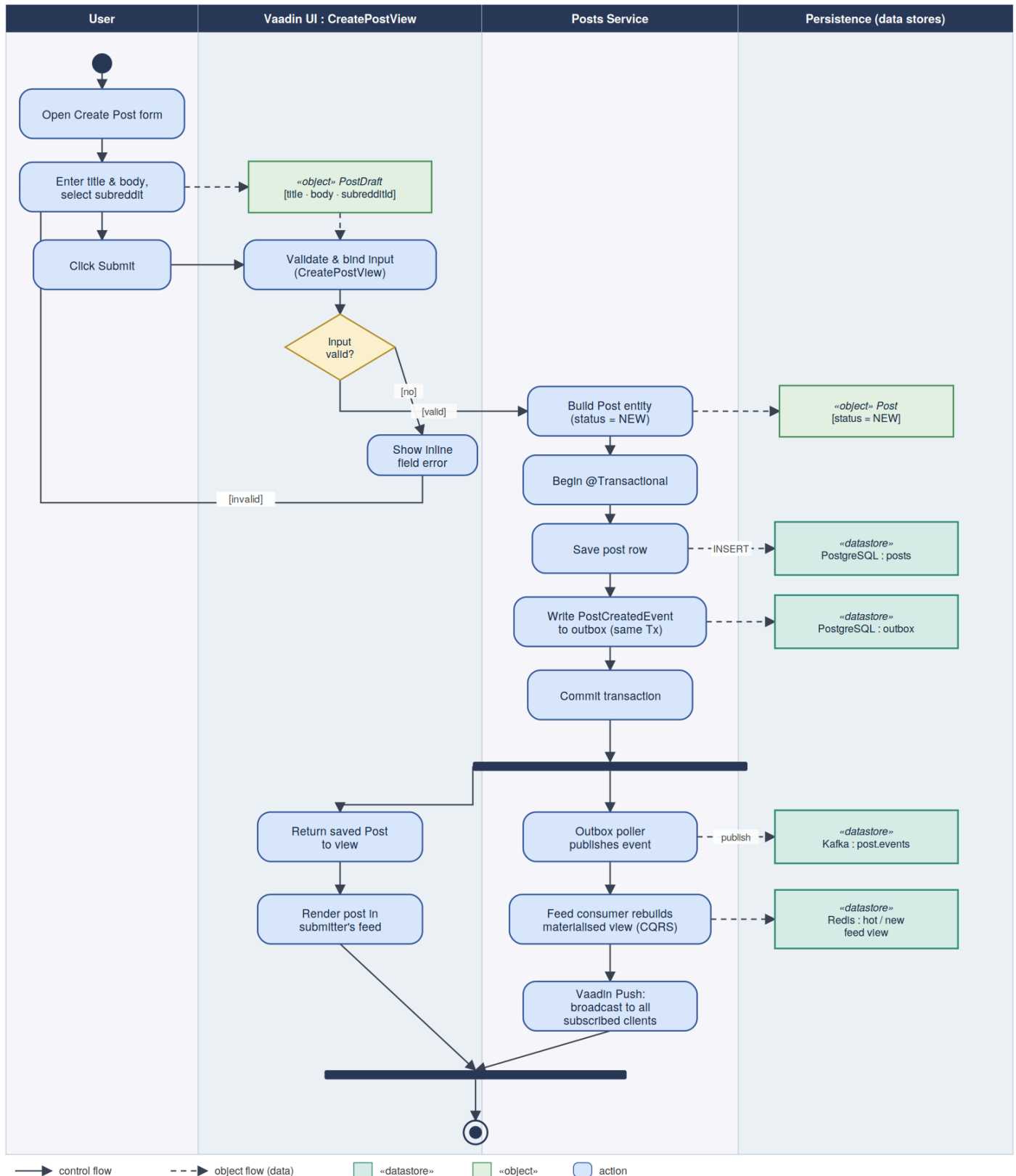


Creating comment



Activity Diagrams

Activity Diagram 1: Create a Post (user interaction + data flow)

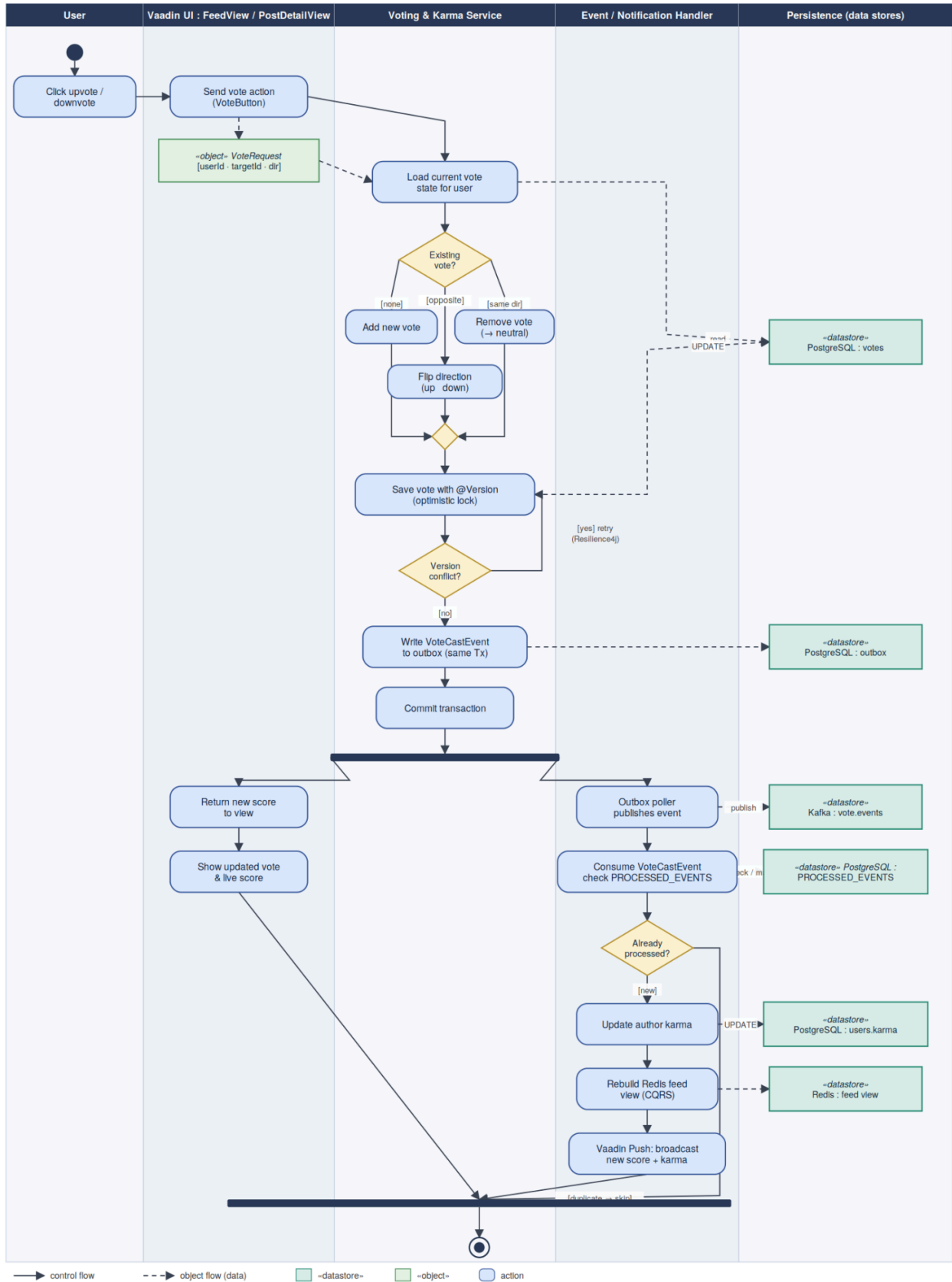


Description:

This activity diagram traces what happens when a registered user creates a post. It starts in the Vaadin UI where the user fills in the title, body, and target subreddit. The CreatePostView validates the input and loops back if something is missing, otherwise the Posts service builds the Post object and saves it. The save and the outbox record are written inside the same transaction, which is what guarantees the event is never lost. After the commit the flow forks: one path returns the saved post straight to the user so they see it right away, while the other runs in the background, where the outbox poller

publishes a PostCreatedEvent to Kafka, a consumer rebuilds the Redis feed view, and Vaadin Push broadcasts the new post to every connected client. The dashed arrows show the data moving between the actions and the data stores.

Activity Diagram 2: Vote on a Post (user interaction + data flow)



Description:

This activity diagram shows how a single vote click ends up updating every user's screen in real time. When the user clicks upvote or downvote, the Voting service loads their current vote state and decides whether to add a new vote, remove an existing one, or flip the direction. The vote is saved with an @Version optimistic lock, and if two people vote at the same instant and cause a version conflict, Resilience4j retries the save instead of dropping the vote. The VoteCastEvent is written to the outbox in the same transaction, and after the commit the flow splits into the immediate response to the voter and the asynchronous event pipeline. On the event side the consumer first checks the PROCESSED_EVENTS table so the same event is never counted twice, then updates the author's karma, rebuilds the Redis feed view, and pushes the new score and karma out to all clients.

Implementation Plan

Approach on Building the Program

We decided to adopt an agile, incremental, feature first approach. Each feature (User, Posts, Voting, Notifications) will be developed from the outside-in starting with the Vaadin UI, then the service layer, then the repository so that each feature becomes demonstrable early. The development of the program is structured in a phased manner as seen in the below table.

Phase	Focus	Deliverable
Phase 1: Foundation	Project setup, shared kernel, Docker compose, CI pipeline	A runnable skeleton with Vaadin + Spring Boot + PostgreSQL
Phase 2: Core User Features	Registration, login, profile, subreddit creation	Users can register, log in, and create subreddits
Phase 3: Content Creation	Posts, comments, basic feeds	Users can create posts, comment, and view a simple feed
Phase 4: Voting & Real-time	Upvote/downvote, optimistic locking, Vaadin Push	Users can vote; vote counts update live across all clients
Phase 5: Advanced Patterns	Kafka, outbox, idempotency, CQRS + Redis, circuit breakers	Event driven notifications; fast materialised feeds
Phase 6: Testing & Polish	Integration tests, load testing, documentation, demo	Fully tested, deployable system

Key Principle: Each phase ends with a working, demonstrable increment that could be shown to the instructor. We do not wait until the end to integrate.

Dependencies

External Dependencies (Tech Stack)

Dependency	Version	Purpose
Java	21+	Runtime environment
Spring Boot	3.2+	Application framework
Vaadin	24.3+	UI framework
PostgreSQL	15+	Primary database
Redis	7+	Caching, materialised views(Session store)

Apache Kafka	3.4+	Event Streaming (Message Broker)
Docker/Docker Compose	Latest	Containerised deployment
Databricks Delta Lake	Latest	Data Lakehouse Format
Apache Spark (embedded)	Latest	Lakehouse Execution Engine
Maven	3.9+	Build tool

Internal Dependencies (Between Features)

Feature	Depends On	Nature of Dependency
Posts	User (authentication)	Only authenticated users can create posts
Comments	Posts, User	Comments belong to a post and a user
Voting	User, Posts, Comments	Votes belong to a user and target post/comment
Notifications	Voting, Comments	Generates events based on voting/comment activity
Feeds (CQRS)	Posts, Voting	Reads from PostgreSQL and Redis; no reverse dependency

Mitigation: We will develop features in dependency order: **User -> Post -> Comments -> Voting -> Notifications**. The shared kernel (Security, event bus) is developed first and used by all.

Development Environment Dependencies

Tool	Purpose
IntelliJ IDEA	IDE
Docker Desktop	Running PostgreSQL, Redis, Kafka locally
Git	Version Control
Postman (Optional)	API testing (IFWAMREST)

Project Planning

Team Structure

Roles & Feature Ownership

Team Member	Primary Feature	Package Location	Secondary Responsibility
Marwan Al-Jaufy	User Context	com.example.readit.user/	Registration, login, JWT, profile management, karma display
Amr Alhamwi	Posts Context	com.example.readit.posts/	Create/edit/delete posts, subreddits, post listing, feed generation
Harsh-Deep Benipal	Comments Context	com.example.readit.comments/	Nested comment threads, edit/delete comments, comment sorting
Farhan Panchbhaya	Voting & Karma	com.example.readit.voting/	Upvote/downvote, optimistic locking (@Version), karma calculation, real time vote push
Mustafa Syed	Notifications & Events	com.example.readit.notifications/	Kafka integration, transactional outbox, idempotency, sagas (post deletion cascade), circuit breakers

Shared Responsibilities (All Team Members)

Responsibility	Owner(s)	Notes
Shared Kernel (Security config, event bus, base entities)	All (rotating weekly lead)	Each member will contribute to the shared code as needed
Docker Compose & deployment	Farhan Panchbhaya (lead) + all	Farhan will own the compose file; everyone runs it locally
CI/CD pipeline (GitHub Actions)	Harsh-Deep Benipal (lead) + all	Setup once; all benefit
Integration testing & Test containers	Each for their own feature	e.g. Farhan tests voting, Amr tests posts etc
Documentation & architecture diagrams	Marwan Al-Jaufy (lead) + All	Marwan will maintain the master document; others will contribute sections
Code reviews	All members	Minimum 2 approvals per pull requests
Vaadin UI consistency	Mustafa Syed (lead)	Ensures common look/feel across all views
Performance & load testing	Amr Alhamwi (lead)	JMeter/Gatling for voting and feed endpoints

Calendar Overview

Milestone	Date	Days Available	Focus
Start	15/06/2026	—	Project kickoff
1st Check-in	26/06/2026	11 days	Foundation + User Context + Posts (basic)
2nd Check-in	10/07/2026	14 days	Comments + Voting + Real-time sync
3rd Check-in	24/07/2026	14 days	Advanced Patterns (Kafka, Outbox, CQRS)
Presentation	29/07/2026	5 days	Polish, demo preparation
Final Report & Deliverables	04/08/2026	6 days	Documentation, video, final submission

Work Breakdown by Phase

Phase 1: Foundation + User Context + Basic Posts (Due: 26/06/2026)

Week	Days	Marwan Al-Jaufy (User)	Amr Alhamwi (Posts)	Harsh-Deep Benipal (Comments)	Farhan Panchbhaya (Voting & Karma)	Mustafa Syed (Notifications & Events)
1	15 - 18 June	Set up shared Kernel (base entity, security placeholder)	Set up Git + GitHub + branch protection	Create Docker Compose (PostgreSQL, Redis, Kafka, Zookeeper)	Create Spring Boot + Vaadin skeleton	Set up Kafka topics (post.events, vote.events, comment.events)
1	18 - 20 June	User registration (Vaadin form)	Configure GitHub Actions CI	Testcontainers setup (base class)	Vaadin Push Configuration (@Push)	Document event schemas
2	20 - 22 June	Login + Spring Security + JWT	Basic post creation (no comments yet)	Assist B with post UI	Optimistic locking setup (@Version on Vote entity)	Prepare outbox table
2	22 - 23 June	User profile view	Basic feed (SQL, no sorting)	Assist A with user UI	Vote entity + repository	Shared Kernel: event bus abstraction
2	24 - 26 June	Karma display (read - only)	Subreddit creation	—	—	—

1st Check-In	26 June	Demo: User can register, log in, view profile	Demo: User can create post, view basic feed	(Supporting role)	(Supporting role)	Demo: Kafka topics exist
--------------	---------	---	---	-------------------	-------------------	--------------------------

Deliverable for 1st Check-In:

- Working user authentication (registration, login, logout)
- User can create a text post
- User can view a basic feed of all posts
- Subreddit creation works
- Docker Compose starts all services
- CI pipeline runs basic tests

Phase 2: Comments + Voting + Real - time synchronization (Due: 10/07/2026)

Week	Days	Marwan Al-Jaufy (User)	Amr Alhamwi (Posts)	Harsh-Deep Benipal (Comments)	Farhan Panchbhaya (Voting & Karma)	Mustafa Syed (Notifications & Events)
3	27 - 30 June	Add Karma calculation (read from voting context)	Post detail view (page for single post)	Basic comment creation (flat, no nesting)	Upvote/down vote logic (posts only)	Add PostCreated Event publisher
3	1 - 3 July	Profile shows comment count	Edit/delete Posts	Comment editing/deletion	Optimistic locking for votes	Add CommentCreatedEvent publisher
4	4 - 7 July	—	Feed sorting (new, top)	Nested comment threads (recursive Vaadin components)	Voting on comments	Add VoteCastEvent publisher
4	8 - 9 July	Real - time Karma update (Vaadin Push)	Real-time post score update	Real - time comment appearance	Vaadin Push broadcast for vote updates	WebSocket broadcaster helper
4	10 July	—	—	—	—	—
2ND CHECK-IN	10 July	Demo: Karma updates live	Demo: Post edit/delete works	Demo: Nested comments work	Demo: Voting works with live updates	Demo: Events published to Kafka

Deliverable for 2nd Check-In:

- Nested comment threads working
- Upvote/downvote working on both posts and comments
- Real-time vote updates (Vaadin Push) across multiple browser windows
- Karma calculation updates in real - time
- Feed sorting (new, top) working
- Events being published to Kafka topics

Phase 3: Advanced Patterns (Due: 24/07/2026)

Week	Days	Marwan Al-Jaufy (User)	Amr Alhamwi (Posts)	Harsh-Deep Benipal (Comments)	Farhan Panchbhaya (Voting & Karma)	Mustafa Syed (Notifications & Events)
5	11 - 14 July	Add user notification preferences UI	CQRS: Materialised feed view in Redis	Comment ranking (best, controversial)	Idempotent vote handler (PROCESSED_EVENTS table)	Transactional Outbox implementation
5	15 - 17 July	Circuit breaker for external (mock) email service	Redis cache configuration (@Cacheable, @CacheEvict)	Comment search (PostgreSQL full - text)	Resilience4j retry for vote persistence	Outbox poller + Kafka publisher
6	18 - 21 July	—	Distributed tracing (Zipkin) for feed queries	Micrometer metrics for comment latency	Load test voting endpoint (JMeter)	Saga: Post deletion cascade (delete post -> delete comments -> delete votes -> compensate on failure)
6	22 - 23 July	Integration tests for user context (Testcontainers)	Integration tests for posts context	Integration tests for comments	Integration tests for voting (with Kafka)	Integration tests for outbox + saga
6	24 July	—	—	—	—	—
3rd Check-In	24 July	Demo: User preferences saved	Demo: Feed loads from Redis (fast)	Demo: Comment sorting works	Demo: Idempotent votes (no duplicates)	Demo: Outbox delivers events after crash; saga works

Deliverable for 3rd Check-In:

- CQRS feed with Radis materialised view (fast feed loading)
- Transactional outbox working (events survive application restart)
- Idempotent vote handling (no duplicate votes)
- Saga pattern for post deletion (with compensation)
- Circuit breakers and retries configured
- Distributed tracing (Zipkin) visible
- Integration tests passing with Test containers

Phase 4: Polish, Testing, Demo Preparation (Due: 29/07/2026)

Days	Marwan Al-Jaufy (User)	Amr Alhamwi (Posts)	Harsh-Deep Benipal (Comments)	Farhan Panchbhaya (Voting & Karma)	Mustafa Syed (Notifications & Events)
25 July	UI polish (responsive layout)	Feed performance tuning	Dark mode toggle	Vote Button debouncing	Final Kafka configuration
26 July	Bug fixes from integration	Bug fixes	Bug fixes	Bug fixes	Bug fixes
27 July	Prepare demo script (user journey)	Create demo data (subreddits, posts, comments)	Test cross - browser (Chrome, Firefox)	Run load test (100 concurrent users)	Final deployment to cloud VM
28 July	Rehearse presentation	Rehearse presentation	Rehearse presentation	Rehearse presentation	Rehearse presentation
29 July	PRESENTATION DAY				

Deliverable for Presentation

- 10 - 15 minute live demo showing all features
- Working cloud deployment (or local with Docker Compose)
- Presentation slides covering architecture, patterns, challenges

